

Оглавление

1. Введение	4
1.1. Исторический контекст	4
1.2. Практическое значение	5
2. Метод ветвей и границ	5
2.1. Теоретические основы	5
2.2. Алгоритм реализации	7
2.3. Оценка сложности	8
3. Метод Tabu Search (поиск с запретами)	9
3.1. Теоретические основы	9
3.2. Детали реализации	10
3.3. Анализ эффективности	11
4. Сравнительный анализ	12
Литература	14

1. Введение

Задача коммивояжера имеет простую формулировку: коммивояжер, выходящий из некоторого города, должен посетить $(n-1)$ других городов и вернуться в исходный пункт. Известны расстояния между всеми городами. Требуется установить, в каком порядке он должен посещать города, чтобы общее пройденное расстояние было минимальным. Если видоизменить постановку задачи и не требовать возвращения к исходному пункту, то получится незамкнутая задача о коммивояжере.

Простота постановки задачи о коммивояжере сочетается с чрезвычайной трудностью ее решения, причем трудности имеют вычислительный характер, так как легко указать прием, прямо ведущий к цели: перебрать все маршруты и взять из них наименьший. Это принципиально возможно, поскольку общее число возможных маршрутов является конечным. Но все дело в том, что ни одна, даже самая быстрая действующая, вычислительная машина не в состоянии осуществить этот перебор при значительном числе городов.

В самом деле, выходя из первого города, коммивояжер может направиться в один из $(n-1)$ городов, откуда — в один из $(n-2)$ остальных городов, и так далее, пока не останется один-единственный город, откуда он вышел. Таким образом, общее число вариантов, подлежащих рассмотрению, составляет $(n-1)!$. С увеличением числа городов это число чрезвычайно быстро возрастает и уже при 15–20 городах достигает огромных значений. Соответственно возрастает и время на перебор этих вариантов, поэтому от полного перебора всех вариантов приходится с самого начала отказаться.

1.1. Исторический контекст

Задача коммивояжера известна с XVIII века. Леонард Эйлер в 1759 году посвятил задаче о шахматном коне, которая тесно связана с данной задачей, большую работу, а в XIX веке Уильям Гамильтон исследовал гамильтоновы циклы на примере головоломки «Икосиан». В 1832 году вышла немецкая книга с практическими советами для коммивояжеров, однако математический аппарат для решения задачи тогда еще не применялся. Первые упоминания как математической задачи на оптимизацию принадлежат Карлу Менгеру (1930 год), который сформулировал ее как «задачу посыльного». Вскоре Хасслер Уитни предложил известное сейчас название «задача странствующего торговца» (англ. travelling salesman problem).

Наряду с простотой формулировки и относительной доступностью получения приемлемых решений задача коммивояжера характеризуется высокой вычислительной сложностью отыскания строго оптимального маршрута. В результате со второй

половины XX века ее исследуют главным образом как теоретическую модель для разработки новых алгоритмов оптимизации, а не для практического применения. На примере задачи коммивояжера были разработаны многие распространенные в настоящее время методы дискретной оптимизации.

1.2. Практическое значение

Задача коммивояжера находит широкое применение в различных областях. В логистике она помогает оптимизировать маршруты доставки, сокращая расход топлива и оптимизируя время. В производстве микросхем — определять порядок сверления отверстий на платах, уменьшая холостой ход оборудования. В астрономии — планировать наведение телескопов для минимизации затрачиваемого времени. Кроме того, данная задача используется при планировании производства, проектировании сетей связи и во многих других сферах для оптимизации затрачиваемых ресурсов.

2. Метод ветвей и границ

2.1. Теоретические основы

Основная идея

Метод ветвей и границ относится к классу точных методов решения задачи коммивояжера. Он удобен как для расчетов вручную, так и для программной реализации.

Суть метода заключается в следующем. Сначала для всего множества возможных гамильтоновых циклов вычисляется нижняя оценка длины маршрута. Затем это множество разбивается на два непересекающихся подмножества: в одном из них содержится заданное ребро, а в другом это ребро исключено. Для каждого из этих подмножеств по аналогичной схеме определяется своя нижняя граница стоимости. При этом новые оценки оказываются не меньше исходной. Сравнивая полученные границы, можно выделить подмножество, которое с наибольшей вероятностью содержит оптимальный маршрут. Выбранное подмножество вновь делится на две части, для которых опять пересчитываются нижние оценки. Этот процесс повторяется до тех пор, пока не будет построен единственный гамильтонов цикл. Вся последовательность разбиений образует дерево решений.

Когда некоторый полный маршрут найден, его длина становится верхней границей для оптимального решения. Далее анализируются ранее отсеченные ветви дерева. Если для какой-либо ветви нижняя оценка оказывается меньше длины уже

найденного маршрута, то эту ветвь необходимо продолжить разбивать, пытаясь найти более короткий путь. В противном случае если нижняя граница ветви больше или равна текущему рекорду, данная ветвь может быть окончательно исключена из дальнейшего рассмотрения, поскольку гарантированно не содержит улучшающего решения.

Нижние оценки и отсечение

Ключевая идея метода ветвей и границ заключается в том, чтобы не перебирать все маршруты, а последовательно сужать множество кандидатов, отсекая заведомо неперспективные подмножества. Для этого каждому подмножеству маршрутов сопоставляется нижняя граница стоимости — число, которое гарантированно не превосходит длины любого маршрута внутри данного подмножества.

Для вычисления нижней границы используется приведение матрицы расстояний путем вычитания минимального элемента из каждой строки и каждого столбца. Сумма всех вычтенных констант называется константой приведения h . В полученной приведенной матрице все элементы неотрицательны, и в каждой строке и каждом столбце есть хотя бы один нуль. Если L — длина оптимального маршрута в исходной задаче, а L_1 — длина оптимального маршрута в задаче с приведенной матрицей, то

$$L = L_1 + h.$$

Поскольку в приведенной матрице все расстояния неотрицательны, любой маршрут имеет длину $L_1 \geq 0$. Следовательно, константа приведения h является нижней границей стоимости оптимального маршрута для рассматриваемого множества.

После того как нижняя граница вычислена, множество маршрутов разбивается на два подмножества по принципу включения или исключения некоторой дуги (i, j) . Для каждого нового подмножества вычисляется приведенная матрица и своя нижняя граница. При этом возникает дерево ветвлений, где каждый узел соответствует подмножеству маршрутов, а его оценка — нижней границе стоимости для этого подмножества.

Если в процессе построения дерева удастся найти некоторый полный маршрут длиной L , то все узлы, у которых нижняя граница $\geq L$, могут быть отсечены — они гарантированно не содержат маршрута короче уже найденного. Таким образом, алгоритм исследует только те ветви, которые потенциально могут улучшить текущий рекорд, что радикально сокращает перебор по сравнению с полным перебором всех $(n - 1)!$ маршрутов.

Выбор дуги для ветвления обычно определяется штрафом за отказ от использования: для каждого нулевого элемента в приведенной матрице вычисляется сумма минимальных элементов в его строке и столбце (кроме самого нуля). Нулевой элемент с максимальным штрафом считается самым критичным: если не включить соответствующее ребро в маршрут, стоимость вырастет сильнее всего. Именно это ребро и выбирается для ветвления. Такой подход позволяет эффективно сужать пространство поиска.

2.2. Алгоритм реализации

Метод ветвей и границ для задачи коммивояжера реализуется как процесс, управляемый очередью с приоритетом. На каждом шаге из очереди извлекается узел с наименьшей нижней границей, после чего он либо приводит к решению, либо разбивается на два дочерних узла. Ниже приведено описание ключевых шагов алгоритма.

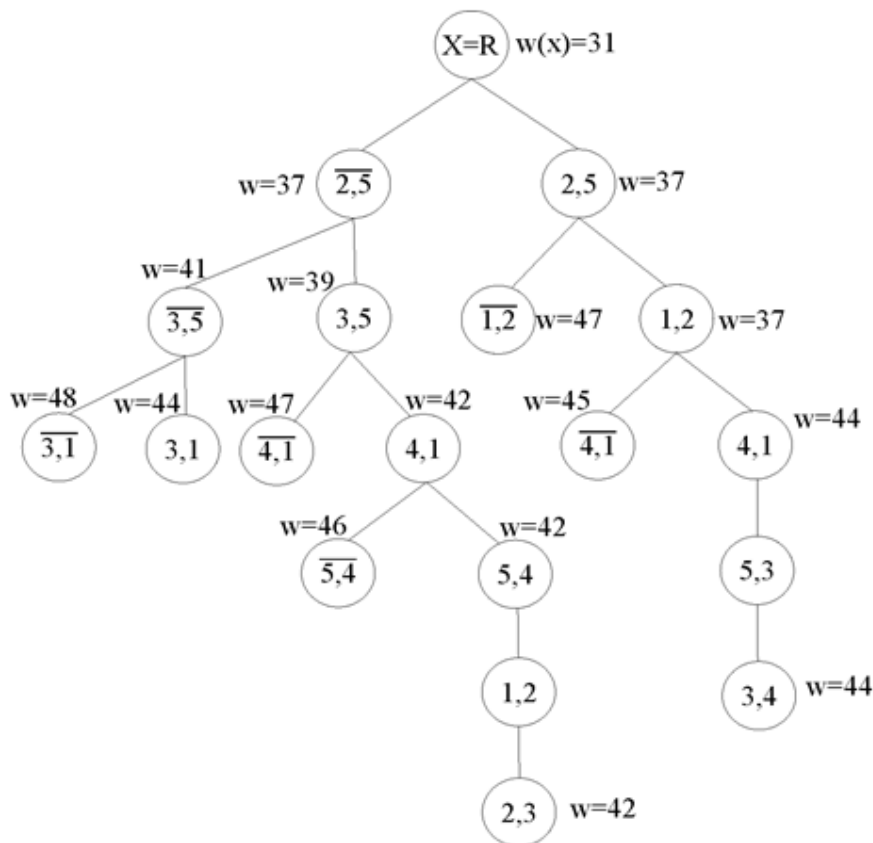


Рис. 2.1. Пример дерева решений метода ветвей и границ [2]

Общая схема

Инициализация. Исходная матрица расстояний приводится — из каждой строки и столбца вычитаются их минимальные элементы. Сумма вычитенных констант образует начальную нижнюю границу. Корневой узел с этой границей помещается в приоритетную очередь.

Выбор узла. Из очереди извлекается узел с наименьшей нижней границей. Если его граница не меньше длины уже найденного маршрута (текущего рекорда), узел отсекается, так как он заведомо не содержит оптимального решения.

Проверка полноты. Если в узле накоплено ровно n ребер (полный гамильтонов цикл), по ним строится маршрут и вычисляется его фактическая стоимость по исходной матрице. Если стоимость меньше рекорда, рекорд обновляется.

Ветвление. В приведенной матрице текущего узла для каждого нулевого элемента вычисляется штраф за неиспользование. Нулевой элемент с максимальным штрафом выбирается для ветвления. По ребру (i, j) создаются два узла. Узел «включить» фиксирует ребро, из матрицы удаляются строка i и столбец j . Узел «исключить» запрещает ребро (i, j) . В обоих случаях матрица приводится заново, а нижняя граница узла увеличивается на соответствующую константу приведения.

Добавление. Если полученная граница дочернего узла оказывается меньше текущего рекорда, узел помещается в очередь для дальнейшего исследования. В противном случае он отбрасывается.

Завершение. Алгоритм заканчивает работу, когда очередь становится пустой. В этот момент текущий рекорд является оптимальным маршрутом, а все неперспективные ветви были отсечены.

2.3. Оценка сложности

В худшем случае метод ветвей и границ может перебрать все $(n - 1)!$ гамильтоновых циклов, что дает теоретическую оценку $O(n!)$. Однако на практике алгоритм работает значительно быстрее за счет механизма отсечений. Ключевая особенность метода состоит в том, что его реальная производительность критически зависит от структуры матрицы расстояний, а не только от числа узлов n .

Для матриц с благоприятной структурой, где нижние границы оказываются достаточно точными, а отсечения происходят на ранних уровнях дерева, алгоритм может найти оптимальное решение, исследовав лишь некоторую часть всех возможных маршрутов. В таких случаях время работы может быть различным на практике. Напротив, для матриц с большим количеством нулевых элементов и малыми штрафами

отсечения происходят реже, и алгоритм вынужден исследовать почти все ветви, приближаясь к теоретическому $O(n!)$.

Таким образом, даже при одинаковом числе узлов время решения двух задач может значительно отличаться из-за разной структуры матриц расстояний: одна решится за доли секунды, другая — за минуты или даже часы.

3. Метод Tabu Search (поиск с запретами)

3.1. Теоретические основы

Метаэвристический подход

Метод поиска с запретами (Tabu Search) был предложен в 1980-х годах Фредом Гловером и относится к классу метаалгоритмов. Метаалгоритм — это процедура или программа, которая управляет другими алгоритмами, комбинирует их, настраивает или оптимизирует.

Классический алгоритм локального поиска обладает существенным недостатком: он останавливается при достижении локального оптимума — решения, в окрестности которого нет ни одного решения с лучшим значением целевой функции. Однако глобальный оптимум также является локальным, поэтому для его обнаружения необходимо уметь переходить от одного локального оптимума к другому.

Рассматриваемый алгоритм решает эту проблему, вводя список запретов. Этот список хранит информацию о недавно посещенных решениях или совершенных перемещениях. При выборе нового решения из окрестности текущего запрещается выбирать решения, содержащиеся в списке запретов. За счет этого алгоритм не возвращается к только что рассмотренным решениям и может покинуть локальный оптимум, продолжая поиск в других областях пространства решений.

Таким образом, поиск с запретами представляет собой итерационный процесс, который на каждом шаге переходит к наилучшему допустимому соседнему решению, даже если оно ухудшает значение целевой функции. Это ключевое отличие от методов локального спуска, которые допускают только улучшающие перемещения. Возможность ухудшения позволяет алгоритму продолжать поиск, не останавливаясь в локальных оптимумах для улучшения результата.

Краткосрочная память

Краткосрочная память реализуется через список запретов. Это структура данных, которая хранит информацию о недавних перемещениях или решениях в тече-

ние определенного числа итераций. Параметр, определяющий максимальный размер списка, называется длиной списка запретов (tabu tenure). При добавлении нового элемента в список, если его длина превышает заданную, из него удаляется самый старый элемент. Краткосрочная память предотвращает заикливание алгоритма и способствует исследованию новых областей пространства решений.

3.2. Детали реализации

Генерация начального решения

В качестве начального решения в программе используется случайный маршрут. Это сделано для того, чтобы алгоритм мог исследовать разные области пространства решений при многократных рестартах.

Локальный поиск

Генерация соседних решений выполняется с помощью операции перестановки двух узлов в маршруте. На каждом шаге перебираются все пары позиций, и для каждой из них формируется кандидат путем перестановки соответствующих узлов. При этом ориентация всех ребер, не инцидентных переставляемым узлам, сохраняется, что позволяет корректно работать с ориентированными графами. Для каждого кандидата проверяются запреты на добавляемые ребра, но если перемещение, улучшающее текущее рекордное решение, запрещено, запрет игнорируется (критерий устремления). Из допустимых кандидатов выбирается маршрут с минимальной стоимостью.

Механизм запретов

Краткосрочная память в реализации представлена списком запретов, который хранит информацию о ребрах, которые временно нельзя использовать. Каждая запись в этом списке содержит информацию о том, какое ребро запрещено, и номер итерации, до которой этот запрет действует.

После каждого перехода к новому решению в список добавляются запреты на те ребра, которые были удалены из маршрута. Запрет действует в течение заданного числа итераций, определяемого параметром длины списка запретов. Проверка запрета выполняется путем просмотра списка: если ребро присутствует в списке и срок его действия не истек, перемещение считается запрещенным.

Очистка просроченных запретов выполняется на каждой итерации: из списка удаляются все записи, у которых истек срок действия. Это гарантирует, что список не

будет неограниченно расти, а запреты будут действовать только в течение заданного временного интервала. Для ориентированного графа используется строгое сравнение по направлению.

3.3. Анализ эффективности

Временная сложность

Временная сложность алгоритма определяется числом итераций, количеством просматриваемых соседей на каждой итерации и стоимостью обработки одного соседа. На каждой итерации алгоритм перебирает все пары позиций в маршруте, то есть $O(n^2)$ соседей, где n — число узлов. Для каждого соседа стоимость не вычисляется заново, а корректируется только по изменившимся рёбрам. Затем проверяется наличие запретов в списке. При фиксированной длине списка запретов эти операции выполняются за $O(1)$. Таким образом, обработка одного соседа требует $O(1)$ операций, а полная итерация имеет сложность $O(n^2)$.

В данной реализации число итераций в одном запуске фиксировано и равно $100 \cdot n$, а количество рестартов составляет $\max(1, n/5)$. Следовательно, общая сложность алгоритма в худшем случае оценивается как $O(n^4)$. Однако на практике благодаря механизмам отсечения (запреты, критерий устремления) реальное число рассматриваемых кандидатов значительно меньше, особенно на поздних итерациях, когда алгоритм уже нашел хорошее решение. Поэтому фактическое время работы существенно ниже теоретической оценки.

Влияние параметров

Эффективность Tabu Search критически зависит от выбора параметров. В данной работе используются следующие эмпирические значения, подобранные на основе различных рекомендаций из литературы.

Длина списка запретов определяет, сколько итераций будут действовать запреты. Этот параметр выбирается пропорционально количеству узлов: чем больше узлов, тем больше должна быть длина списка, чтобы эффективно предотвращать заикливание. При слишком коротком списке алгоритм может быстро возвращаться к только что посещенным решениям, что приводит к заикливанию. При слишком длинном списке большая часть окрестности оказывается запрещенной, что ограничивает способность алгоритма исследовать новые области.

Максимальное число итераций определяет, сколько шагов алгоритм будет выполнять в одном запуске. Это значение выбирается так, чтобы обеспечить до-

статочное количество шагов для сходимости, но не быть избыточным для задач с небольшим числом узлов.

Количество итераций без улучшений определяет, сколько итераций подряд алгоритм может работать без улучшения лучшего решения. Ранняя остановка при отсутствии улучшений — стандартный прием, позволяющий экономить вычислительные ресурсы, когда алгоритм оказывается в локальном оптимуме, из которого не может выбраться.

Число рестартов определяет, сколько раз алгоритм запускается с разных начальных маршрутов. Многократные запуски повышают вероятность нахождения глобального оптимума, так как каждый раз алгоритм исследует новые области пространства решений.

Таким образом, выбор параметров представляет собой компромисс между качеством решения и временем выполнения, и зависит от конкретной постановки задачи и доступных вычислительных ресурсов.

4. Сравнительный анализ

Таблица 1. Сравнение методов (реализация сrr)

Метод	Тип	n	Время (с)	Стоимость
Tabu Search	Несим.	4	0.006931	145
Tabu Search	Несим.	10	0.019591	156
Tabu Search	Несим.	20	0.325613	747
Tabu Search	Несим.	30	2.521721	542
Tabu Search	Сим.	4	0.006591	90
Tabu Search	Сим.	10	0.017378	146
Tabu Search	Сим.	20	0.408257	733
Tabu Search	Сим.	30	2.011023	466
Ветви и границы	Несим.	4	0.000277	145
Ветви и границы	Несим.	10	0.003984	156
Ветви и границы	Несим.	20	0.056439	738
Ветви и границы	Несим.	30	—	—
Ветви и границы	Сим.	4	0.001294	90
Ветви и границы	Сим.	10	0.029387	146
Ветви и границы	Сим.	20	0.058994	723
Ветви и границы	Сим.	30	—	—

Реализация Tabu Search на языке программирования C++ выигрывает у реализации на Wolfram Language за счет двух факторов. Во-первых, cpp — компилируемый язык, и операции над массивами выполняются напрямую, без интерпретации. Во-вторых, в C++ список запретов и матрица расстояний хранятся в непрерывной памяти, и доступ к ним осуществляется за $O(1)$, тогда как в Wolfram Language проверка запрета по списку требует линейного просмотра и создания промежуточных выражений. Как следствие, при $n = 30$ время работы на C++ составило 2 секунды, а на Wolfram — более 400 секунд.

Таблица 2. Сравнение методов (реализация Wolfram Mathematica)

Метод	Тип	n	Время (с)	Стоимость
Tabu Search	Несим.	4	0.000522	145
Tabu Search	Несим.	10	0.531826	156
Tabu Search	Несим.	20	34.158025	738
Tabu Search	Несим.	30	425.031253	542
Ветви и границы	Несим.	4	0.001001	145
Ветви и границы	Несим.	10	0.028333	156
Ветви и границы	Несим.	20	0.484249	738
Ветви и границы	Несим.	30	—	—

Выбор между методами определяется размерностью задачи и требованиями к точности. Метод ветвей и границ гарантирует нахождение глобального оптимума, но применим лишь для малых размерностей ($n \leq 20$). Tabu Search не дает гарантии оптимальности найденного решения, однако позволяет получать удовлетворительные решения за приемлемое время для задач большой размерности.

Литература

1. Задача коммивояжёра / свободная энциклопедия «Википедия»
URL: https://ru.wikipedia.org/wiki/Задача_коммивояжёра (дата обращения: 11.08.2026).
2. Метод ветвей и границ / Задача коммивояжера / Хабр
URL: <https://habr.com/ru/articles/560468/> (дата обращения: 11.08.2026).
3. Копылова Е.С., Николаева Д.С., Бунтова Е.В. Решение задачи коммивояжёра с использованием метода ветвей и границ / Human Progress. — 2018. — Т. 4, № 4.
URL: http://progress-human.com/images/2018/Tom4_4/Kopylova.pdf (дата обращения: 11.08.2026).
4. Поиск с запретами / Оптимизационные алгоритмы / Простейшие задачи размещения / Дискретные задачи размещения
URL: http://old.math.nsc.ru/AP/benchmarks/UFLP/uflp_tabu.html (дата обращения: 11.08.2026).
5. Поиск с запретами / свободная энциклопедия «Википедия»
URL: https://ru.wikipedia.org/wiki/Поиск_с_запретами (дата обращения: 11.08.2026).
6. Поиск с запретами / энциклопедия «Знания Вики»
URL: https://ru.ruwiki.ru/wiki/Поиск_с_запретами (дата обращения: 03.04.2025).